



Unidad 3: Procesamiento de Datos XML

Introducción a la Programación y Computación 2

Escuela de Ingeniería en Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Agenda

RECORDATORIOS

- ¿Qué es XML?
- Partes y Características
- Ventajas y Desventajas
- Aplicaciones
- Procesamiento de Datos XML

Recursos

Competencias que Desarrollaremos

Analiza actividades en archivos XML utilizando técnicas de lectura estructurada para identificar y extraer información relevante



¿Qué es XML?



Es un metalenguaje que fue diseñado para estructurar, almacenar e intercambiar datos entre aplicaciones. Es un estándar ya que es extensible y puede ser utilizado independientemente de la plataforma.

Es el acrónimo de Extensible Markup Language. Es un lenguaje de marcado que define un conjunto de reglas para la codificación de documentos.

Es un metalenguaje que permite definir lenguajes de marcas desarrollado por el World Wide Web Consortium (W3C) utilizado para almacenar datos de forma legible.

¿Para qué nos sirve XML?

Brindar una estructura a la información que se encuentre en la web,
para que la información pueda ser:

Almacenada

Procesada

Visualizada

Manipulada

Por aplicaciones, páginas o lo que lo requiere.



Partes de Xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<biblioteca>
  <libro>
    <titulo>La vida está en otra parte</titulo>
    <autor>Milan Kundera</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Pantaleón y las visitadoras</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Conversación en la catedral</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1969"/>
  </libro>
</biblioteca>
```

Prólogo: En este se especifica que el documento en cuestión es XML, se define la versión y la codificación de caracteres que se utilizó en el archivo

Elementos: Se tiene una etiqueta o tag inicial <> y una final </>.

Epílogo: Datos adicionales, como encriptación. Generalmente los tokens son agregados en este apartado. Este elemento es opcional.

Estructura de una Etiqueta



Características de XML

- **Extensible:** Cada persona puede crear las etiquetas que desee según el tipo de documento que se quiere crear. En otras palabras cada usuario podrá crear su propio lenguaje.
- **Estándar abierto:** XML es SGML, lo que significa que no es necesario saber programar, además de que existen muchas herramientas para su manipulación.



Características de XML

- **Eficiente:** La información debe ser transmitida una única vez para su manipulación.
- **Libre:** Nadie tiene una patente sobre él, ya que ha sido definido como un estándar internacional.
- **Manejable:** Tiene métodos para declarar y reforzar las estructuras que son utilizadas actualmente.



Ventajas de XML

- **Simplifica el intercambio de datos:** Dada la gran cantidad de aplicaciones que se tienen en la web, los desarrolladores pueden utilizar el mismo estándar para comunicar aplicaciones que a simple vista serían incompatibles.
- Los archivos XML se almacenan en forma de texto, lo que facilita la expansión y actualización de los mismos.



Ventajas de XML

- **Aumenta la disponibilidad de datos:** El acceso a los archivos XML pueden darse de diferentes lugares, no únicamente desde HTML.
- Con XML se tiene la opción que los datos estén disponibles para todos los tipos de máquinas de lectura.



Desventajas de XML

- **Verboso:** XML tiende a ser más verboso que JSON, lo que puede hacer que los documentos sean más grandes y difíciles de leer.
- **Procesamiento más lento:** Debido a su estructura más compleja, el procesamiento de datos XML puede ser más lento en comparación con JSON.



Desventajas de XML

- **Tamaño de archivo mayor:** En comparación con JSON, los documentos XML tienden a ser más grandes debido a su sintaxis y a la necesidad de incluir etiquetas de apertura y cierre para cada elemento. Esto puede llevar a un aumento en el consumo de ancho de banda y tiempo de transferencia, lo que podría ser una consideración importante en aplicaciones donde la velocidad de transferencia es crucial, como en servicios web o aplicaciones móviles.



Aplicaciones XML

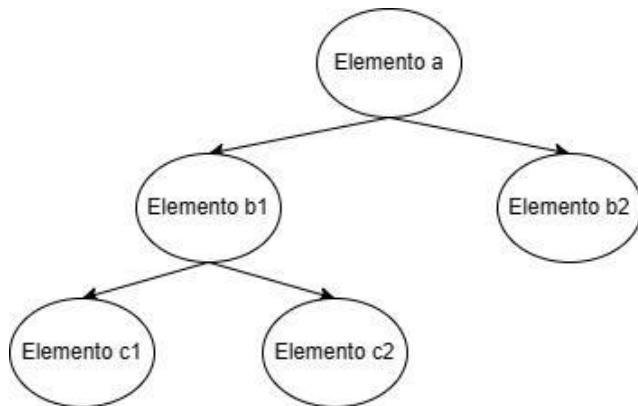
- Estructura HTML
- PDF está guardado en XML
- Microsoft Office guarda los datos como XML
- SOAP(Simple Object Access Protocol)
- Web Services de instituciones financieras
- Correos electrónicos



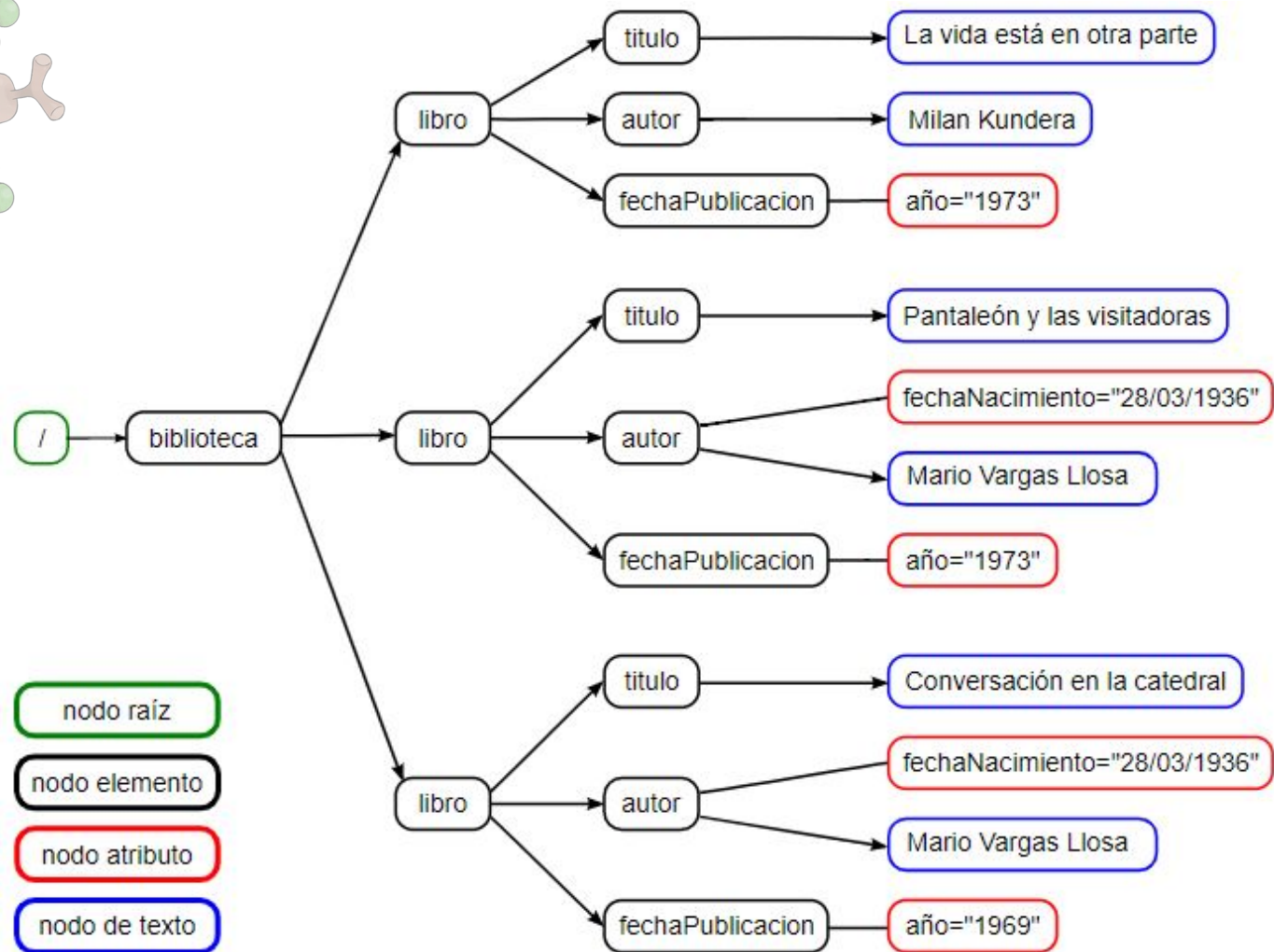
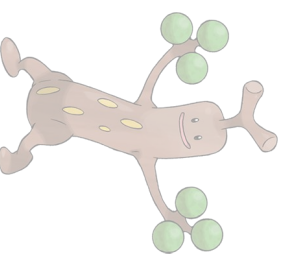
Árbol

Un árbol es una estructura jerárquica o secuencial formada por nodos, los cuales pueden ser de dos tipos: los nodos hijos y los nodos padre.

- Un padre puede tener muchos hijos
- Un hijo puede tener sus propios hijos, además de tener hermanos



```
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
  <libro>
    <titulo>La vida está en otra parte</titulo>
    <autor>Milan Kundera</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Pantaleón y las visitadoras</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Conversación en la catedral</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1969"/>
  </libro>
</biblioteca>
```

Procesamiento de datos XML

En Python usamos:



Modelo DOM

El Modelo de Objetos del Documento, o DOM por sus siglas en inglés, es una librería del World Wide Web (W3C) para acceder y modificar documentos XML. Una implementación del DOM presenta los documento XML como un árbol, y permite al código cliente construir dichas estructuras desde cero para Crear nuestros propios árboles XML.

```
from xml.dom.minidom import parse, parseString

dom1 = parse('c:\\temp\\mydata.xml') # parse an XML file by name

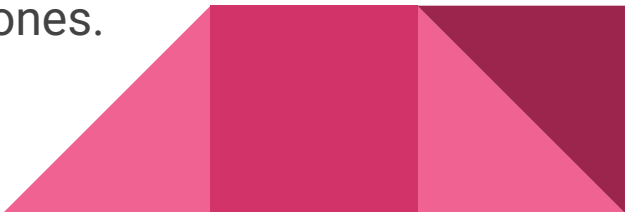
datasource = open('c:\\temp\\mydata.xml')
dom2 = parse(datasource) # parse an open file

dom3 = parseString('<myxml>Some data<empty/> some more data</myxml>')
```

En el DOM, los documentos tienen una estructura lógica que es muy parecida a un árbol, sin embargo, no especifica que debe ser implementado como un árbol obligatoriamente.

El DOM es un modelo lógico que puede implementarse de cualquier manera que sea conveniente.

Una propiedad importante de los modelos de estructura del DOM es algo llamado “isomorfismo estructural”, el cual significa que si dos implementaciones cualesquiera del Modelo de Objetos del Documento se usan para crear una representación del mismo documento, ambas crearán el mismo modelo de estructura, con exactamente los mismos objetos y relaciones.



Origen del modelo DOM

El DOM se originó como una especificación para permitir que los programas Java y los scripts de JavaScript fueran portables entre navegadores web.

La primera especificación del modelo surgió en 1997 llamada DOM level 1 del cual se realizó una actualización en 2000.

Se han realizado nuevas especificaciones al modelo agregando más funcionalidades para el procesamiento de archivos con propiedades que necesitan de otras funciones.



Origen del modelo DOM

Los elementos que componen al archivo XML son tratados por el modelo DOM como objetos. Los atributos que contienen estos objetos son tratados por el model DOM como cadenas de caracteres o strings, por lo que tenemos que tomarlo en cuenta al realizar operaciones con los datos recién obtenidos del archivo de entrada.

Los atributos en el modelo DOM también pueden ser tratados como nodos en vez de strings aunque es bastante inusual hacerlo.



Objetos en el modelo DOM

Interfaz	Sección	Propósito
DOMImplementation	Objetos DOMImplementation	Interfaz para las implementaciones subyacentes.
Node	Objetos Nodo	Interfaz base para la mayoría de objetos en un documento.
NodeList	Objetos NodeList	Interfaz para una secuencia de nodos.
DocumentType	Objetos DocumentType	Información acerca de la declaraciones necesarias para procesar un documento.

Objetos en el modelo DOM

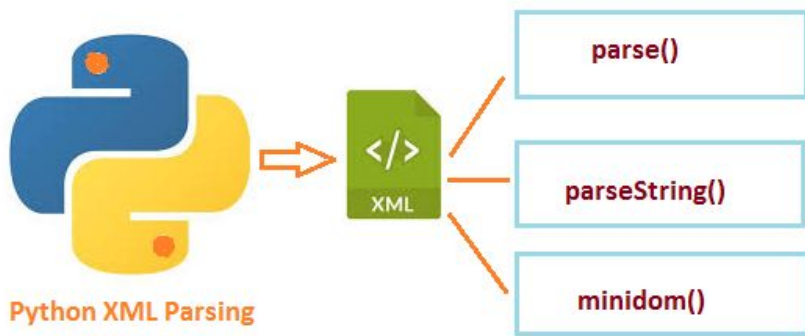
Document	Objetos Documento	Objeto que representa un documento entero.
Element	Objetos Elemento	Nodos elemento en la jerarquía del documento.
Attr	Objetos Atributo	Nodos de los valores de los atributos en los elementos nodo.
Comment	Objetos Comentario	Representación de los comentarios en el documento fuente.
Text	Objetos Texto y CDATASection	Nodos con contenido textual del documento.
ProcessingInstruction	Objetos ProcessingInstruction	Representación de instrucción del procesamiento.

MiniDOM

MiniDOM o Minimal DOM Implementation es una implementación simplificada del modelo DOM que puede utilizarse en Python.

Para utilizarlo es necesario importarlo desde el módulo xml.dom.

Este modelo usa la función “parse” para crear un objeto DOM directamente desde nuestro archivo XML.



MiniDOM

La función tiene la siguiente sintaxis:

```
xml.dom.minidom.parse(filename_or_file(parser(bufsize)))
```



How to Parse XML in Python

ScrapingAnt

MiniDOM

La propiedad “file_or_filename” debe contener una ruta hacia el archivo XML o hacia un objeto de tipo archivo. Esta función retorna un documento el cual ya podremos procesar como un XML.

Para empezar a buscar elementos con el nombre específico de una tag utilizamos el siguiente comando:

getElementByTagName()



How to Parse XML in Python

ScrapingAnt

MiniDOM

Debido a que cada nodo se trata como un objeto, podemos acceder a los valores de los tags usando las propiedades de los objetos.

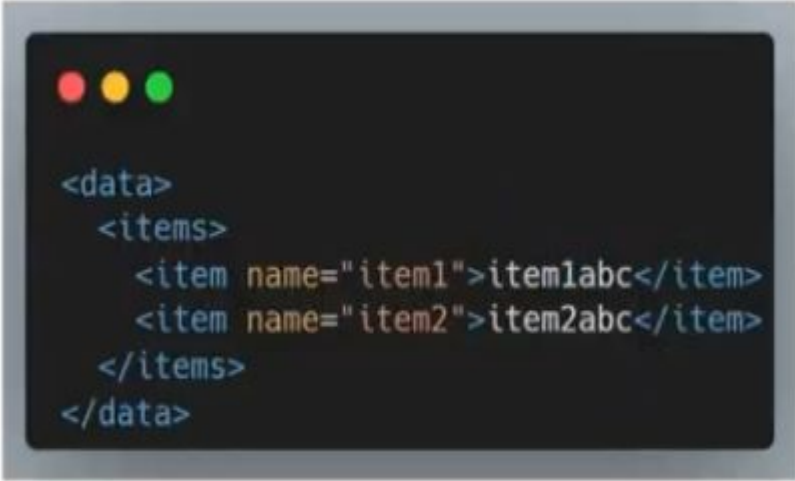
Por ejemplo siguiente podremos observar el procesamiento de un archivo XML.



How to Parse XML in Python

ScrapingAnt

Ejemplo de archivo de XML de entrada



```
<data>  
  <items>  
    <item name="item1">item1abc</item>  
    <item name="item2">item2abc</item>  
  </items>  
</data>
```

Pasos para leer con MiniDOM

01

Primero debemos importar la librería:

```
1 #importing minidom library
2 from xml.dom import minidom
3
```

02

Mediante el comando “parse” creamos un objeto documento con la ruta del archivo XML.

```
4 #parsing XML file
5 xmldoc = minidom.parse('D:/ProjectPro/Tutorials/How to parse XML in python/items.xml')
6
```


Pasos para leer con MiniDOM

03

Para obtener los nodos con el tag "item" utilizamos el comando "getElementsByTagName"

```
7  #getting list of elements
8  itemlist = xmldoc.getElementsByTagName('item')
9
```

04

Podemos obtener los valores de los atributos del tag de la siguiente manera:

```
10 #printing number of elements
11 print("Number of elements: ",len(itemlist))
12
13 #printing name of first element
14 print("Name of First element:", itemlist[0].attributes['name'].value)
15
```

Pasos para leer con MiniDOM

05

Ahora bien, si lo que se desea es obtener todos los valores posibles dentro del tag específicamente, también podemos usar instrucciones cíclicas de la siguiente manera:

```
16 #printing name of all the elements
17 print("Name of all the elements:")
18 for s in itemlist:
19     print(s.attributes['name'].value)
```

Element Tree para procesar XML

ElementTree es una biblioteca estándar de Python para leer, escribir y manipular documentos XML. Es una API ligera y fácil de usar que te permite trabajar con archivos XML de forma eficiente.

Características:

- Fácil de usar: Ofrece una API intuitiva y fácil de aprender.
 - Ligera: No requiere la instalación de bibliotecas adicionales.
 - Eficiente: Permite leer y escribir archivos XML de forma rápida.
 - Flexible: Soporta la manipulación de la estructura del árbol XML.
 - Compatible con XPath: Permite realizar consultas XPath para seleccionar nodos específicos.
- 

Escribir documentos XML con ElementTree

ElementTree también es ideal para escribir datos en archivos XML. El siguiente código muestra cómo crear un archivo XML con la misma estructura que el archivo que usamos en los ejemplos anteriores.

1. Cree un elemento, que actuará como nuestro elemento raíz. En este caso se llamará “data”

```
import xml.etree.ElementTree as ET  
  
# create the file structure  
data = ET.Element('data')
```

2. Una vez que tenemos nuestro elemento raíz, podemos crear subelementos usando la SubElement función sintaxis:

```
SubElement(parent, tag, attrib={}, **extra)
```

```
items = ET.SubElement(data, 'items')
```

Escribir documentos XML con ElementTree

Aquí parent está el nodo principal al que conectarse, attrib hay un diccionario que contiene los atributos del elemento y extra hay argumentos de palabras clave adicionales. Esta función nos devuelve un elemento, que puede usarse para adjuntar otros subelementos, como lo hacemos en las siguientes líneas al pasar elementos al SubElement constructor.

3. Aunque podemos agregar nuestros atributos con la SubElement función, también podemos usar la set() función, como hacemos en el siguiente código. El texto del elemento se crea con la propiedad text del Elemento objeto.

```
item1 = ET.SubElement(items, 'item')
item2 = ET.SubElement(items, 'item')
item1.set('name', 'item1')
item2.set('name', 'item2')
item1.text = 'item1abc'
item2.text = 'item2abc'
```

Escribir documentos XML con ElementTree

4. En las últimas 3 líneas del código siguiente, creamos una cadena a partir del árbol XML y escribimos esos datos en un archivo que abrimos.

La ejecución de este código dará como resultado un nuevo archivo, "items2.xml", que debería ser equivalente al archivo "items.xml" original, al menos en términos de la estructura de datos XML. Probablemente notará que la cadena resultante es solo una línea y no contiene sangría,

```
# create a new XML file with the results  
mydata = ET.tostring(data)  
myfile = open("items2.xml", "w")  
myfile.write(mydata)
```

Encontrar elementos

El ElementTree módulo ofrece la `findall()` función, que nos ayuda a encontrar elementos específicos en el árbol. Devuelve todos los artículos con la condición especificada. Además, el módulo tiene la función `find()`, que devuelve solo el primer subelemento que coincide con los criterios especificados.

Sintaxis:

```
findall(match, namespaces=None)
```

```
find(match, namespaces=None)
```

Para ambas funciones, el `match` parámetro puede ser un nombre de etiqueta XML o una ruta. La función `findall()` devuelve una lista de elementos y `find` devuelve un solo objeto de tipo `Element`.



Encontrar elementos

se recorren todos los elementos de l root y todos los subElementos 'item'

1. Obtenemos el dict de cada elemento para obtener el nombre del atributo en este caso 'name'
2. Obtenemos únicamente el nombre de cada elemento si ya conocemos el atributo nombrado 'name'

```
for elem in root:
    for subelem in elem.findall('item'):

        # if we don't need to know the name of the attribute
        print(subelem.attrib)

        # if we know the name of the attribute
        print(subelem.get('name'))
```



```
{'name': 'item1'}
item1
{'name': 'item2'}
item2
```

Aplicar de lo aprendido



Conceptos claves aprendidos

- ¿Qué es XML.
- Partes de XML (Prólogo, elementos y epílogo).
- Estructura de una etiqueta.
- Características de XML.
- Modelo Dom
- Element Tree



Ética e Integridad

📌 Ejemplo:

*Al procesar datos XML, especialmente personales o financieros, es clave actuar con **ética** e **integridad**, respetando la privacidad y el uso correcto de la información.*

📌 Pregunta para analizar:

¿Qué harías si tuvieras acceso a datos personales en XML? ¿Cómo aplicarías estos valores?



Referencias

- DesarrolloWeb.com. (s.f.). *Manual de XML*. Recuperado el 24 de mayo de 2025, de <https://desarrolloweb.com/manuales/manual-xml.html>
- AprendeAProgramar.com. (s.f.). *Introducción a XML*. Recuperado el 24 de mayo de 2025, de https://www.aprendeaprogramar.com/index.php?option=com_content&view=article&id=483
- MDN Web Docs. (s.f.). *XML*. Mozilla Foundation. Recuperado el 24 de mayo de 2025, de <https://developer.mozilla.org/es/docs/Web/XML>



¡GRACIAS POR SU ATENCIÓN!

DUDAS



**RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA**

